

Backend gas64 : troncature des arguments de taille octet

Rapport de bogue et correctif proposé — FreeBASIC 1.20.0, cible win64 —
src/compiler/ir-gas64.bas

1. Résumé

Lors de la préparation d'un appel, un argument de taille **octet** ou **mot** (BYTE, UBYTE, BOOLEAN, CHAR, SHORT, USHORT) est écrit *uniquement dans la partie basse* du registre d'argument. Les bits supérieurs conservent la valeur précédente du registre. Si la fonction appelée lit un type plus large — ce qui est le cas de tout prototype C déclarant `int` — elle lit des données résiduelles.

Le backend **gcc** n'est pas affecté. Le bogue ne dépend pas de la fonction appelée : il concerne toute valeur d'un octet ou de deux octets transmise à un appelé qui en lit davantage.

2. Symptôme observé

La fonction `FB.Split` ignorait son indicateur de sensibilité à la casse : elle se comportait toujours comme si celui-ci valait *vrai*, y compris lorsqu'il était explicitement mis à *false*.

```
Split( "aBxC", "x" )           ' gcc : 1      gas64 : 3
Split( "aBxC", "x", true )      ' gcc : 3      gas64 : 3
Split( "aBxC", "x", false )     ' gcc : 1      gas64 : 3
```

La troisième ligne est la plus parlante : la valeur `false` est transmise explicitement et reste sans effet.

3. Reproduction minimale

Trois conditions sont nécessaires, et aucune ne suffit seule. La fonction appelante doit renvoyer un UDT (ce qui introduit le pointeur de résultat caché, donc décale les registres d'arguments), transmettre un paramètre `byref` et un paramètre `byval` d'un octet, à un appelé externe `"C"`.

```
extern "C"
  declare function SS alias "fb_StrSplitSpans" _
    ( byref s as const string, byref delim as const ustring, _
      byval ic as boolean, byval outp as integer ptr, _
      byval maxpairs as integer ) as integer
end extern

private function V1( byref s as const string, byref delim as const ustring, _
  byval ic as boolean ) as Array( of long )
  dim r as Array( of long )
  r.Push( SS( s, delim, ic, 0, 0 ) ) ' gcc : 1      gas64 : 3
  return r
end function
```

4. Diagnostic : le code produit

Code généré par gas64 pour l'appel ci-dessus :

```
V1:
  mov  QWORD PTR 16[rbp], rcx      ; pointeur de résultat caché
  mov  QWORD PTR 24[rbp], rdx      ; s
  mov  QWORD PTR 32[rbp], r8       ; delim
  mov  BYTE  PTR 40[rbp], r9b      ; ic
  ...
  mov  rcx, 24[rbp]                ; arg 1 = s
  mov  rdx, 32[rbp]                ; arg 2 = delim
  mov  r8b, 40[rbp]                ; arg 3 = ic  <-- OCTET DE POIDS FAIBLE SEUL
  xor  r9d, r9d                    ; arg 4 = NULL
  mov  QWORD PTR 32[rsp], 0        ; arg 5 = 0
```

```
call fb_StrSplitSpans
```

L'instruction `mov r8b, 40[rbp]` n'écrit que 8 bits. Or r8 contenait encore le pointeur de `lim`, reçu dans ce même registre à l'entrée de la fonction. Le registre vaut donc l'adresse de `lim` dont seul l'octet de poids faible a été remplacé.

Le prototype C est `FBCALL ssize_t fb_StrSplitSpans( FBSTRING *s, FBSTRING *delim, int ic, ssize_t *out, ssize_t maxpairs )`. L'appelé lit `int ic`, c'est-à-dire les 32 bits de poids faible de r8 : une portion d'adresse, pratiquement toujours non nulle, donc interprétée comme *vrai*.

5. Pourquoi le problème restait invisible

Situation	Code produit	Résultat
Argument littéral (false / true)	<code>xor r8d, r8d</code> (registre entier remis à zéro)	correct
Argument 32 bits (LONG)	<code>mov r8d, ...</code> (écrire un registre 32 bits met à zéro la moitié haute sur x86-64)	correct
Argument 8 ou 16 bits depuis une variable	<code>mov r8b, ...</code> <code>mov r8w, ...</code>	défectueux

Un argument littéral est émis avec une remise à zéro du registre complet ; c'est pourquoi presque tous les appels de la bibliothèque, qui passent une constante, fonctionnaient. Seuls les cas où la valeur provient d'une variable ou d'un paramètre transmis exposent le défaut.

6. Origine dans le code

Dans `hdocal1( )`, sélection de l'instruction selon le type de l'argument :

```
case FB_DATATYPE_INTEGER, ...
    asm_code("mov " + *regstrq(listreg(cptint)) + ", " + op1)    ' 64 bits : correct
case FB_DATATYPE_LONG, FB_DATATYPE_ULONG
    asm_code("mov " + *regstrd(listreg(cptint)) + ", " + op1)    ' 32 bits : correct
case FB_DATATYPE_SHORT, FB_DATATYPE_USHORT
    asm_code("mov " + *regstrw(listreg(cptint)) + ", " + op1)    ' 16 bits : DÉFECTUEUX
case FB_DATATYPE_BYTE, FB_DATATYPE_UBYTE, FB_DATATYPE_BOOLEAN, FB_DATATYPE_CHAR
    asm_code("mov " + *regstrb(listreg(cptint)) + ", " + op1)    ' 8 bits : DÉFECTUEUX
```

Le cas 32 bits est correct par une particularité de x86-64 : écrire un registre de 32 bits met automatiquement à zéro la moitié haute du registre de 64 bits. Cette garantie n'existe ni pour les écritures de 8 bits ni pour celles de 16 bits.

7. Correctif proposé

Étendre la valeur à l'ensemble du registre au lieu de n'en écrire qu'une partie, en respectant le signe : `movsx` pour `BYTE` et `SHORT` (signés), `movzx` pour `UBYTE`, `USHORT`, `BOOLEAN` et `CHAR`. Écrire dans le registre de 32 bits suffit, puisque cela met à zéro la moitié haute.

```
case FB_DATATYPE_BYTE
    if v2->typ = IR_VREGTYPE_REG then
        asm_code("movsx " + *regstrd(listreg(cptint)) + ", " + op1)
    else
        asm_code("movsx " + *regstrd(listreg(cptint)) + ", BYTE PTR " + op1)
    end if
case FB_DATATYPE_UBYTE, FB_DATATYPE_BOOLEAN, FB_DATATYPE_CHAR
    if v2->typ = IR_VREGTYPE_REG then
        asm_code("movzx " + *regstrd(listreg(cptint)) + ", " + op1)
    else
        asm_code("movzx " + *regstrd(listreg(cptint)) + ", BYTE PTR " + op1)
    end if
```

Les cas `SHORT` et `USHORT` sont traités de la même manière avec `WORD PTR`. La distinction `IR_VREGTYPE_REG` est nécessaire car `movzx` et `movsx` exigent un préfixe de taille pour un opérande mémoire, mais pas pour un registre.

Code produit après correction :

```
mov    rcx, 24[rbp]
mov    rdx, 32[rbp]
movzx  r8d, BYTE PTR 40[rbp]      ; registre entièrement défini
xor     r9d, r9d
mov     QWORD PTR 32[rsp], 0
call   fb_StrSplitSpans
```

8. Vérification

Après correction, `gas64` et `gcc` donnent des résultats identiques sur le cas d'origine ainsi que sur la reproduction minimale. La suite de tests de `fb` a été exécutée sous les deux backends : 1 155 921 assertions, 2 398 modules, plus 1 746 tests de compilation-exécution répartis sur quatre dialectes, et les fichiers de référence des diagnostics sur cinq cibles. Aucun écart n'a été introduit.

Avant correction, la suite comptait sous `gas64` un échec de plus que sous `gcc` (`string_.fbstr_split`) ; c'était la seule manifestation détectée du défaut.

9. Portée

Le défaut n'est pas propre à `Split`. Il touche tout argument de 8 ou 16 bits transmis depuis une variable ou un paramètre à une fonction qui en lit davantage — en pratique, tout paramètre `boolean` d'une déclaration `extern "C"` dont le prototype indique `int`. Les conséquences sont silencieuses : aucun avertissement, aucune erreur, seulement une valeur erronée reçue par l'appelé.

Document rédigé à partir d'un diagnostic mené sur FreeBASIC 1.20.0 (cible win64). Le code assembleur cité est reproduit tel qu'émis par le compilateur, avant et après modification.